KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

함수 3
lambda
map, filter

짧은 함수는 이름 없이 만들고
리스트 전체에 한 번에 쓰기

```
prices = [4500, 5000, 6500]

list(map(lambda p: p + 500, prices))
# [5000, 5500, 7000]

list(filter(lambda p: p >= 5000, prices))
# [5000, 6500]
```

오늘의 순서

1	왜 필요한가	되풀이하는 for와 append, map과 filter 그림, 가장 짧은 map	4–6쪽
2	함수를 값처럼, lambda	함수 담기와 넘기기, def와 lambda 비교, 바로 넘기기	7–11쪽
3	map	for 네 줄을 한 줄로, 하나씩 넣는 그림, map object, str과 len 넘기기	12–15쪽
4	filter	골라 담기를 한 줄로, True만 남는 그림, 딕셔너리 키, map과 이어 쓰기	16–19쪽
5	오류와 함정	번호와 len, 한 번만 꺼내지는 map, 괄호, return, 긴 lambda	20–24쪽
6	실습	10% 인상 가격표, 품절 아닌 메뉴, 튜플 주문 기록	25–27쪽

3쪽은 노트북 준비, 28–29쪽은 막힐 때와 한 장 요약이에요.

들어가기

20에서 이어서 basics.ipynb에 적어요

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 basics.ipynb 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 Shift+Enter로 실행

기억이 안 나는 줄은 오른쪽 번호의 챕터를 먼저 다시 봐요.

앞에서 배운 것

리스트, 튜플, for, append 13

딕셔너리 키로 for 돌기 15

if, 한 줄 if else 17

def, return, 괄호를 뺀 hello 19

키워드 인자 reverse=True 20

1. 왜 필요한가

가격을 바꿀 때마다 for 네 줄을 다시 적어요

[1]
```
prices = [4500, 5000, 6500]
up = []
for p in prices:
    up.append(p + 500)      ← 500원 올리기
print(up)
down = []
for p in prices:
    down.append(p - 500)    ← 500원 내리기
print(down)
```

← 나머지는 똑같아요

← 나머지는 똑같아요

```
[5000, 5500, 7000]
[4000, 4500, 6000]
```

이번 챕터에서 배우면 한 줄이에요

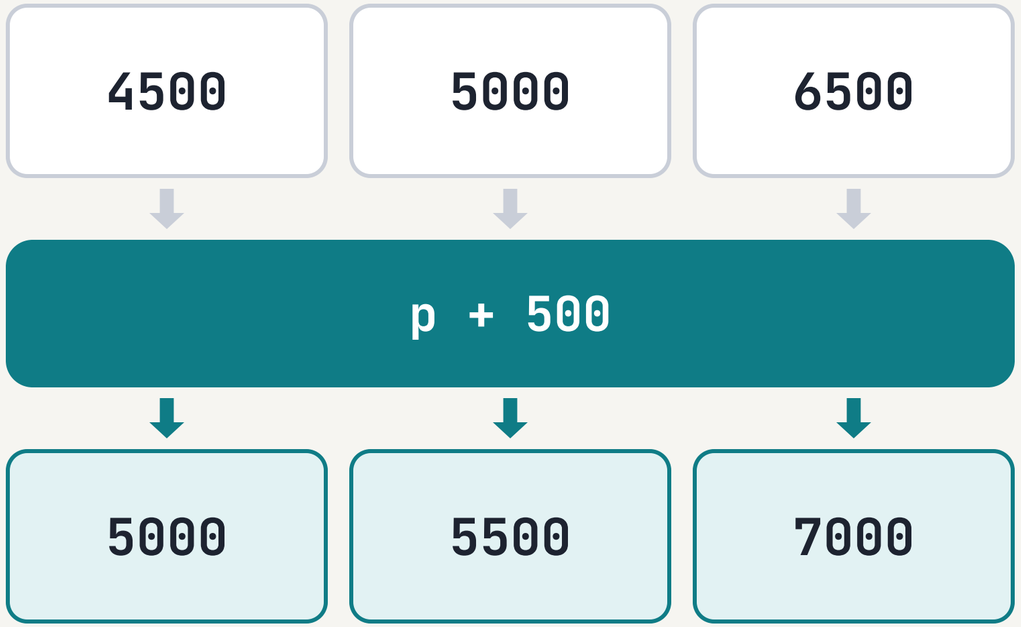
```
up = list(map(lambda p: p + 500, prices))
```

두 덩어리에서 다른 건 한 줄뿐이에요.

1. 왜 필요한가

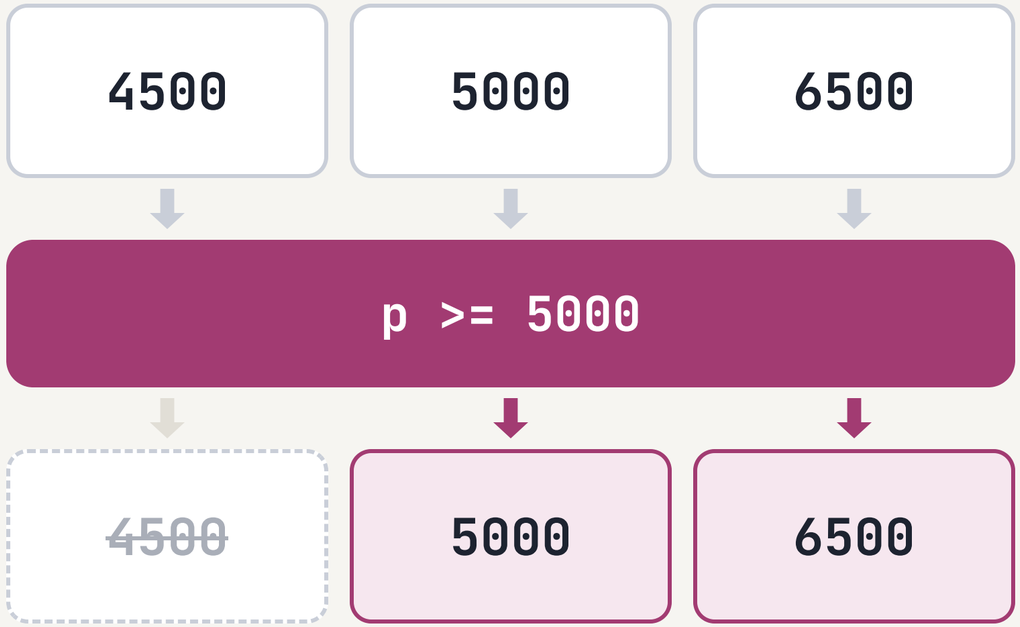
map은 하나씩 바꾸고, filter는 골라내요

`map(함수, 리스트)` 값마다 바꿔요



결과도 3개예요

`filter(함수, 리스트)` 골라내요



True인 값만 남아서 2개예요

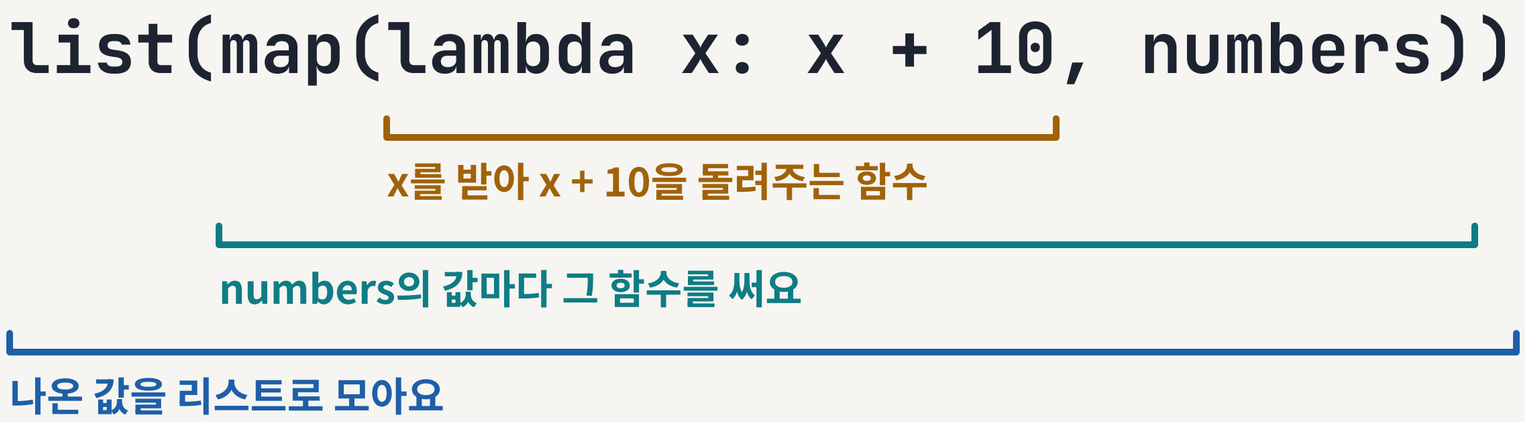
가운데 띠에 넣는 짧은 함수를 lambda로 만들어요.

1. 왜 필요한가

가장 짧은 map: 값마다 10 더하기

[3]
```python
numbers = [1, 2, 3]
print(list(map(lambda x: x + 10, numbers)))
```

```
[11, 12, 13]
```



함수와 리스트를 map에 넘기면, 값마다 함수를 써서 결과를 모아 줘요.

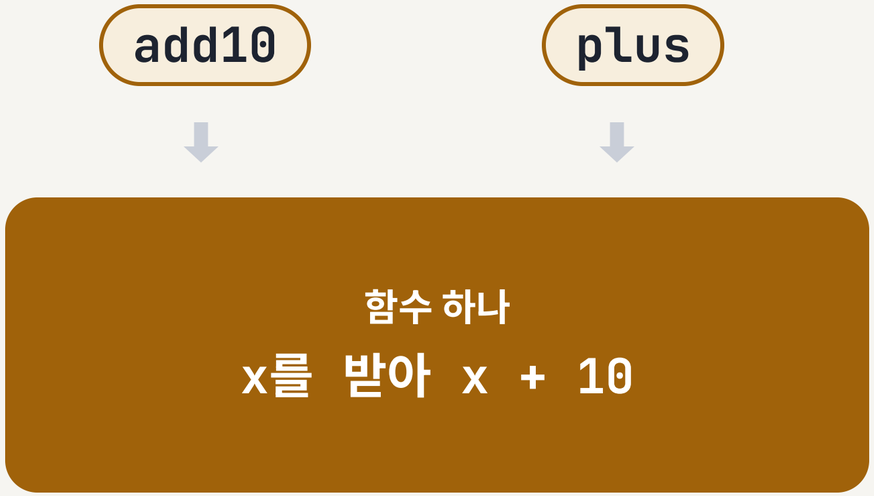
2. 함수를 값처럼, lambda

함수도 변수에 담을 수 있어요

[1]
```python
def add10(x):
    return x + 10

plus = add10          ← 괄호 없이 담아요
print(plus(5))
print(add10)
```

```
15
<function add10 at 0x7fdb817ce4b0>
```



같은 함수에 이름이 하나 더 생겨요

괄호를 빼면 함수를 실행하지 않고 값처럼 다룰 수 있어요.

함수를 다른 함수에 넘길 수 있어요

[2]
```python
def double(x):
    return x * 2

def apply(f, x):
    return f(x)          ← 받은 함수를 불러요

print(apply(add10, 5))
print(apply(double, 5))  ← 이름만 넘겨요
```

Python

```
15
10
```

apply(add10, 5)

➡ `add10(5)` ➡ 15

apply(double, 5)

➡ `double(5)` ➡ 10

함수도 인자로 넘길 수 있어요. 넘길 때는 이름만 적어요.

2. 함수를 값처럼, lambda

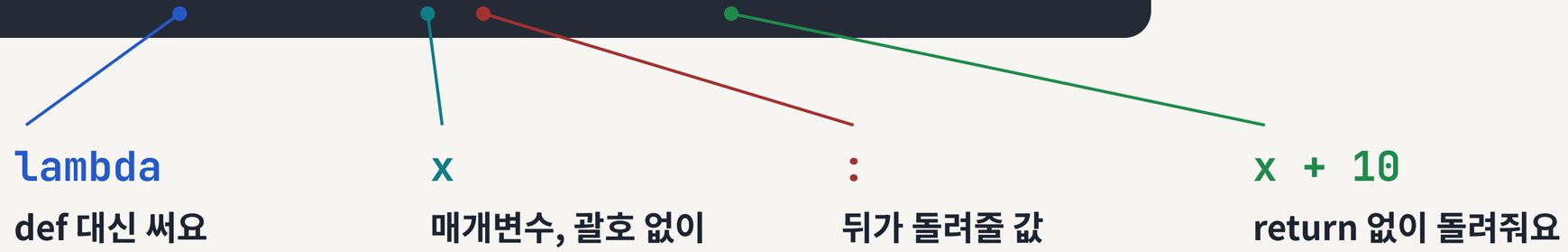
lambda는 한 줄로 만드는 이름 없는 함수예요

19의 def

```
def add10(x):
    return x + 10
```

lambda

```
lambda x: x + 10
```



이름, 괄호, return을 빼고 매개변수와 돌려줄 값만 남긴 모양이에요.

2. 함수를 값처럼, lambda

lambda는 다른 함수에 바로 넘길 때 써요

변수에 담아 부르기

[3]
```
add10 = lambda x: x + 10
print(add10(5))
print(add10)
```
Python

```
15
<function <lambda> at 0x7fdb817cf270>
```
← 이름 자리가 비어 있어요

이름 없이 바로 넘기기

[4]
```
print(apply(lambda x: x * 3, 5))
```
← 앞 장의 apply에 그 자리에서 만들어 넘겨요

Python

```
15
```

한 번 쓰고 말 짧은 함수는 lambda로 그 자리에 적어요.

2. 함수를 값처럼, lambda

매개변수 여러 개, 한 줄 if else도 돼요

[5]

```
price_of = lambda price, qty: price * qty          ← 쉼표로 둘
print(price_of(4500, 2))
label = lambda p: "비쌈" if p >= 5000 else "보통"   ← 17의 한 줄 if else
print(label(6500), label(4500))
```

Python

```
9000
비쌈 보통
```

	def	lambda
이름	있어요	없어요
몸통	여러 줄	값 하나
return	적어요	안 적어요
언제	여러 번 부를 함수	다른 함수에 바로 넘길 때

3. map

for 네 줄이 map 한 줄이 돼요

[1]
```python
prices = [4500, 5000, 6500]

def add500(p):
    return p + 500

print(list(map(add500, prices)))                ← def로 만든 이름 넘기기
print(list(map(lambda p: p + 500, prices)))     ← lambda 바로 넘기기
print(prices)                                   ← 원본은 그대로
```

Python

```
[5000, 5500, 7000]
[5000, 5500, 7000]
[4500, 5000, 6500]
```

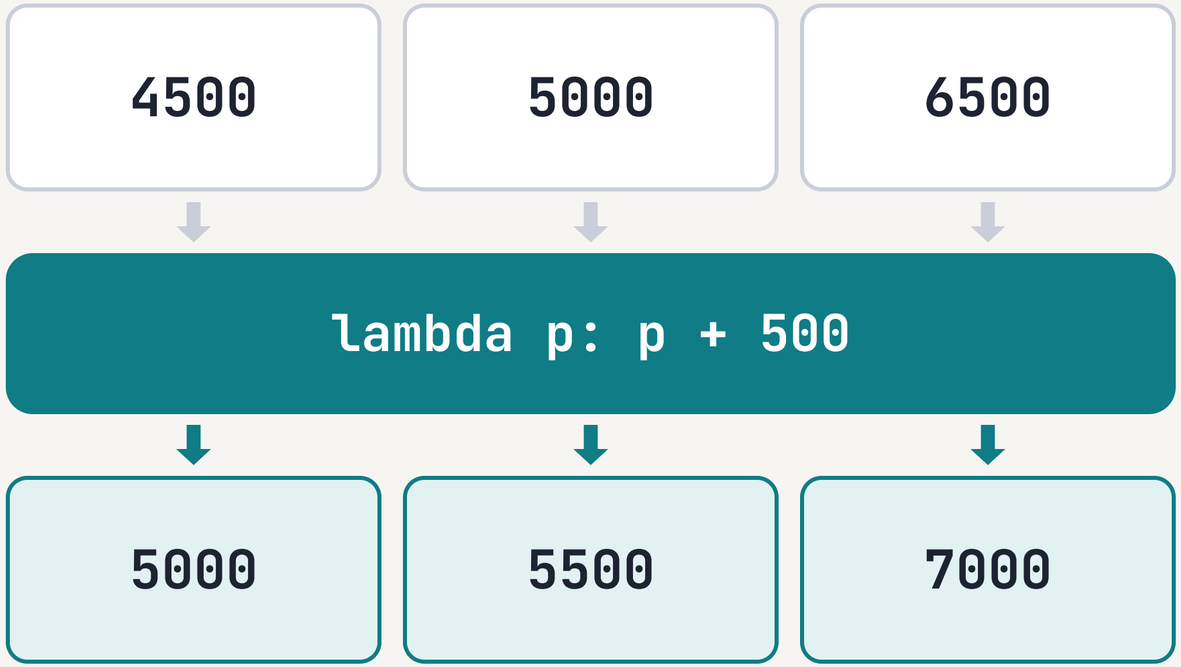
1파트의 for도 같은 결과예요

```python
up = []
for p in prices:
    up.append(p + 500)
```

map(함수, 리스트)는 for와 append로 새 리스트를 만드는 일을 한 줄로 줄여요.

3. map

map은 값을 하나씩 함수에 넣어요



1 p = 4500 ➡ 5000

2 p = 5000 ➡ 5500

3 p = 6500 ➡ 7000

넣은 순서대로 3개가 나와요

map이 리스트의 값을 차례로 lambda의 p에 넣고, 돌려받은 값을 모아요.

3. map

map만 찍으면 <map object>가 나와요

[2]
```python
m = map(lambda p: p + 500, prices)
print(m)
print(list(m))
```

```
<map object at 0x7fe9b3214840>
[5000, 5500, 7000]
```

← 값을 꺼내 줄 준비만 한 묶음이에요

← list()가 끝까지 꺼내 모아요

[3]
```python
for p in map(lambda p: p + 500, prices):
    print(p)
```

← for도 하나씩 꺼내 써요

```
5000
5500
7000
```

결과를 리스트로 보려면 list()로 감싸요.

3. map

이미 있는 함수도 이름만 넘겨요

[4]

```
print(list(map(str, prices)))
print(", ".join(map(str, prices)))
print(list(map(int, ["3", "1", "2"])))
print(list(map(len, ["coffee", "latte", "tea"])))
```

← str: 숫자를 글자로
← join 앞에서 글자로 바꾸기
← int: 글자를 숫자로
← len: 값마다 길이 재기

Python

```
['4500', '5000', '6500']
4500, 5000, 6500
[3, 1, 2]
[6, 5, 3]
```

str, int, len처럼 이미 있는 함수로 되는 일은 lambda 없이 이름만 넘겨요.

4. filter

for와 if로 골라 담던 일을 filter 한 줄로

[1]

```
prices = [4500, 5000, 6500, 3000]
big = []
for p in prices:
    if p >= 5000:
        big.append(p)
print(big)
print(list(filter(lambda p: p >= 5000, prices)))
```

Python

← 17에서 for 안의 if로 골라 담기

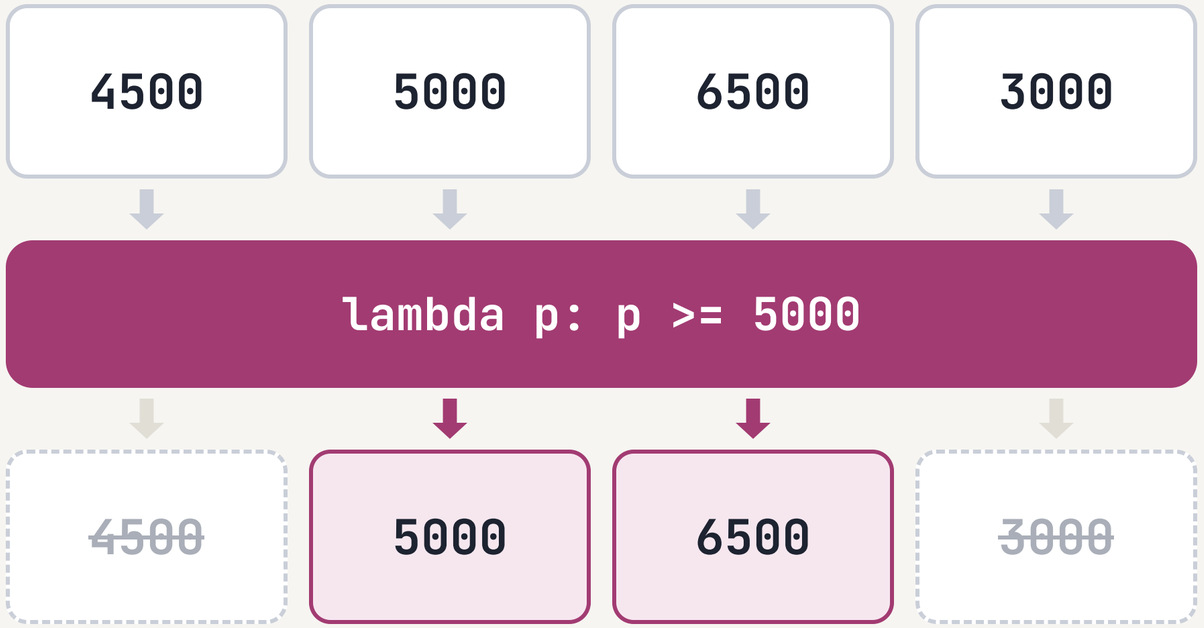
```
[5000, 6500]
[5000, 6500]
```

← 결과가 같아요

for 안의 if로 골라 담는 일은 filter 한 줄로 줄여요.

4. filter

True를 받은 값만 아래로 내려가요



check = lambda p: p >= 5000으로 하나씩 불러 본 결과

p	check(p)	filter는
4500	False	버려요
5000	True	남겨요
6500	True	남겨요
3000	False	버려요

>=라서 5000도 True예요

filter에 넘기는 함수는 True나 False를 돌려주는 조건이에요.

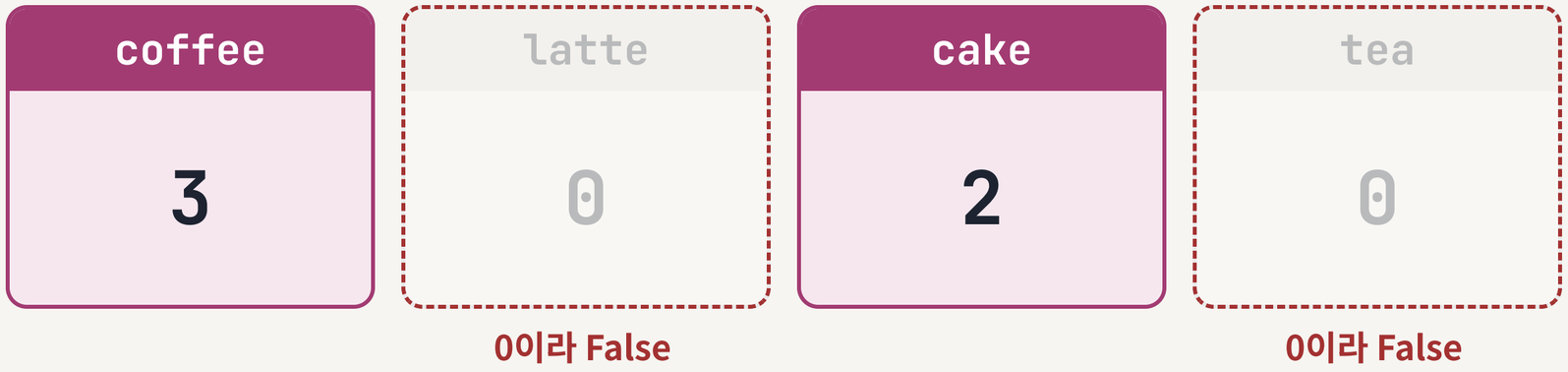
4. filter

딕셔너리를 넘기면 키가 하나씩 들어가요

[3]
```python
stock = {"coffee": 3, "latte": 0, "cake": 2, "tea": 0}
print(list(filter(lambda m: stock[m] > 0, stock)))
```

```
['coffee', 'cake']
```

← m에는 키가 들어가고, 재고는 stock[m]으로 봐요



`['coffee', 'cake']` 키만 담긴 리스트예요. 재고 숫자는 없어요.

딕셔너리를 넘기면 키로 골라서, 조건에 맞는 키만 남겨요.

4. filter

고른 다음 바꾸려면 filter 결과를 map에 넘겨요

[4]
```python
big = list(filter(lambda p: p >= 5000, prices))
print(big)
print(list(map(lambda p: p * 9 // 10, big)))
print(list(map(lambda p: p * 9 // 10, filter(lambda p: p >= 5000, prices))))
```
← 두 번에 나눠 쓰면 중간 결과 big을 볼 수 있어요

Python

```
[5000, 6500]
[4500, 5850]
[4500, 5850]
```

```
list(map(lambda p: p * 9 // 10, filter(lambda p: p >= 5000, prices)))
```

① 안쪽 filter가 먼저 골라요

② 바깥 map이 고른 값을 바꿔요

고르고 바꾸기를 같이 할 땐 filter로 고른 결과를 map에 넘겨요.

5. 오류와 함정

map 결과에는 번호도 len도 못 써요

[1]
```python
doubled = map(lambda x: x * 2, [1, 2, 3])
print(doubled[0])
```

```
TypeError: 'map' object is not subscriptable
```

[2]
```python
print(len(doubled))
```

```
TypeError: object of type 'map' has no len()
```
← filter 결과도 같아요

[3]
```python
doubled = list(map(lambda x: x * 2, [1, 2, 3]))
print(doubled[0])
print(len(doubled))
```
← list()로 바꾸면 돼요

```
2
3
```

번호로 꺼내거나 길이를 재려면 먼저 list()로 바꿔요.

map 결과는 한 번 꺼내면 비어요

[4]
```python
m = map(lambda p: p + 500, [4500, 5000])
print(list(m))
print(list(m))
```

```
[5000, 5500]
[]
```

← 다 꺼낸 뒤라 비었어요

첫 번째 list(m) → 5000 5500

두 번째 list(m) → [] 남은 값이 없어요

이렇게 고쳐요

[5]
```python
m = map(lambda p: p + 500, [4500, 5000])
ups = list(m)
print(ups)
print(ups)
```

```
[5000, 5500]
[5000, 5500]
```

두 번 이상 쓸 결과는 list()로 바꿔 변수에 담아 둬요.

넘길 함수에 괄호를 붙이면 바로 실행돼요

[6]
```
def add10(x):
    return x + 10

print(list(map(add10(), [1, 2, 3])))
```
← map보다 먼저 x 없이 실행돼요

Python

```
TypeError: add10() missing 1 required positional argument: 'x'
```

× ~~map(add10(), [1, 2, 3])~~ → ✓ `map(add10, [1, 2, 3])`

함수를 넘길 땐 괄호 없이 이름만 적어요.

lambda 안에 return을 쓰면 SyntaxError가 나요

```
[7] add10 = lambda x: return x + 10    ← 콜론 뒤에는 값만 와요
                                                          Python
    SyntaxError: invalid syntax
```

~~× lambda x: return x + 10~~ → ✓ `lambda x: x + 10`

lambda는 콜론 뒤에 돌려줄 값 하나만 적어요.

5. 오류와 함정

lambda가 길어지면 def로 바꿔요

lambda 한 줄

[8]
```python
label = lambda p: "비쌈" if p >= 6000 else ("보통" if p >= 4500 else "쌈")
print(label(6500), label(5000), label(3000))
```

```
비쌈 보통 쌈
```

def로 나눠 쓰기

[9]
```python
def label(p):
    if p >= 6000:
        return "비쌈"
    elif p >= 4500:
        return "보통"
    return "쌈"

print(label(6500), label(5000), label(3000))
```

← 조건이 한 줄에 하나씩

```
비쌈 보통 쌈
```

← 결과는 같아요

6. 실습

메뉴 가격을 10% 올린 표를 만들어요

[1]

```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
print(list(map(lambda m: menu[m] * 11 // 10, menu)))  ← 새 가격만

def line(m):
    new = menu[m] * 11 // 10                          ← 줄 만들기는 길어서 def
    return f"{m:<7}{menu[m]:>6,}원 → {new:>6,}원"

print("\n".join(map(line, menu)))                     ← 메뉴마다 한 줄, \n으로 이어 찍기
```

Python

```
[4950, 5500, 7150]
coffee  4,500원 →  4,950원
latte   5,000원 →  5,500원
cake    6,500원 →  7,150원
```

한 줄 계산은 lambda, 줄 만들기처럼 긴 일은 def로 만들어 map에 넘겨요.

6. 실습

품절이 아니고 5000원 미만인 메뉴를 골라요

[2]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500, "tea": 4000}
stock = {"coffee": 3, "latte": 0, "cake": 2, "tea": 5}
print(list(filter(lambda m: stock[m] > 0, menu)))
print(list(filter(lambda m: stock[m] > 0 and menu[m] < 5000, menu)))  ← 17의 and
```
Python

```
['coffee', 'cake', 'tea']
['coffee', 'tea']
```

m	stock[m] > 0	menu[m] < 5000	둘 다
coffee	True	True	True
latte	False	False	False
cake	True	False	False
tea	True	True	True

조건이 두 개면 lambda 안에서 and로 이어요.

6. 실습

튜플로 적은 주문 기록을 고르고 꺼내요

[3]

```python
orders = [("kim", "latte", 2), ("lee", "cake", 1), ("park", "coffee", 3), ("kim", "tea", 1)]
many = list(filter(lambda o: o[2] >= 2, orders))   ← 2잔 이상 산 주문만 골라요
print(many)
print(list(map(lambda o: o[0], many)))             ← 고른 주문에서 이름만 꺼내요
print(sum(map(lambda o: o[2], orders)))            ← 잔 수만 꺼내 sum에 바로 넘겨요
```

Python

```
[('kim', 'latte', 2), ('park', 'coffee', 3)]
['kim', 'park']
7
```

o 칸에 들어오는 튜플 하나



튜플 하나가 lambda의 o 칸에 들어가고, 번호로 칸을 꺼내요.

정리

막히면 오류의 마지막 줄을 봐요

	보이는 것	까닭	고치기
1	<map object at 0x...>가 나와요	map 결과를 그대로 찍었어요	`list(map(...))`
2	`TypeError: 'map' object is not subscriptable`	map 결과에 번호를 붙였어요	**list()로 바꾸기**
3	`TypeError: object of type 'map' has no len()`	map 결과에 len을 썼어요	**list()로 바꾸기**
4	두 번째 list(m)이 []예요	map은 한 번 꺼내면 비어요	**리스트로 담아 두기**
5	`TypeError: add10() missing 1 required ...`	넘길 함수에 괄호를 붙였어요	`map(add10, xs)`
6	`SyntaxError: invalid syntax`	lambda 안에 return을 적었어요	`lambda x: x + 10`
7	lambda가 한눈에 안 읽혀요	조건이 여러 개예요	**def로 만들어 넘기기**

결과가 이상하면 list()로 바꿔 중간 결과를 찍어 봐요.

한 장 요약

`lambda x: x + 10`	이름 없는 한 줄 함수	`map(str, xs)`	있는 함수는 이름만
`plus = add10`	괄호 없이 함수 담기	`filter(f, xs)`	True인 값만 남겨요
`map(f, xs)`	값마다 f를 써요	`filter(f, d)`	딕셔너리는 키가 들어가요
`list(map(...))`	리스트로 모으기	`def label(p):`	길어지면 def로 넘기기

함수를 넘길 땐 괄호 없이 이름만 적어요.

다음 챕터 22에서는 sorted, max, min의 key에 lambda를 넣어 줄 세울 기준을 정해요.